Лабораторная работа №7.
Docker Compose. Многоконтейнерные приложения

4 академических часа • Debian 13 • PostgreSQL 18

Тема	Docker Compose: Nginx, Express API и PostgreSQL 18 с переиспользованием объектов ЛР №5–6
Цель	Научиться описывать уже созданные контейнерные компоненты через Docker Compose, использовать существующие Docker Network и Volume, добавить Nginx reverse proxy и подтвердить сохранность данных PostgreSQL.

1. Общие сведения

Лабораторная работа №7 продолжает сквозной проект ЛР №5 и ЛР №6. В предыдущих работах уже созданы PostgreSQL 18 с постоянным томом, пользовательская Docker-сеть и Express API. В этой работе их имена и данные не меняются: Docker Compose переводит тот же учебный стек под декларативное управление и добавляет Nginx как reverse proxy.

Главный принцип работы: не создавать новый PostgreSQL-кластер, новую сеть или новый том. Сначала проверяем существующие объекты, затем удаляем только старые контейнеры pg-course и api-net6 и создаём их через Compose с прежними именами, образами, сетью и томом.

2. Тема и цель

Тема: Docker Compose для многоконтейнерного приложения Nginx → Express API → PostgreSQL 18. Цель: описать сервисы в compose.yaml, подключить существующие внешние ресурсы course-net и pgdata-course18, сохранить имена контейнеров pg-course и api-net6, проверить сквозной HTTP/SQL-запрос и убедиться, что данные таблицы categories сохраняются после docker compose down/up.

3. Планируемые результаты

1. Объяснять структуру compose.yaml и назначение services, networks, volumes, depends_on и healthcheck.
2. Проверять конфигурацию командой docker compose config до запуска.
3. Переиспользовать существующие Docker Network и Volume как external-ресурсы.
4. Сохранять сквозные имена контейнеров pg-course и api-net6.
5. Использовать существующие образы postgres:18, express-pg:net6 и nginx:stable.
6. Настроить Nginx как reverse proxy к api-net6:3000.
7. Проверять реальное обращение API к таблице categories в базе shop.
8. Диагностировать ошибку DNS-имени и остановку PostgreSQL, затем восстанавливать сервис.

4. Исходные данные и оборудование

Понадобится Debian 13 с Docker Engine и Docker Compose v2, терминал с sudo и файлы, созданные в предыдущих лабораторных. Интернет обычно не нужен, если все необходимые образы уже загружены. Перед началом работы обязательно проверить существование объектов сквозного проекта.

Компонент	Каноническое имя / значение
Папка Compose-проекта	~/compose-course7
Сеть Docker	course-net (существующая, external)
Образ PostgreSQL	postgres:18
Контейнер PostgreSQL	pg-course
Volume PostgreSQL	pgdata-course18 (существующий, external)
Точка монтирования Volume	/var/lib/postgresql
PGDATA PostgreSQL 18	/var/lib/postgresql/18/docker
База данных	shop
Пользователь PostgreSQL	student
Учебный пароль	Lab5-ChangeMe-2026
Образ Express API	express-pg:net6
Контейнер Express API	api-net6
PGHOST из API	pg-course

Компонент	Каноническое имя / значение
Образ Nginx	nginx:stable
Новый контейнер Nginx	nginx-course7
HTTP на хосте	127.0.0.1:8080 → nginx-course7:80
Маршрут проверки	GET /api/db-check

ВАЖНО	Не создавайте новый Docker Volume, новую Docker-сеть и новый PostgreSQL-кластер. В этой работе используются существующие pgdata-course18 и course-net. Не выполняйте docker volume rm, docker volume prune или docker system prune.

5. Схема взаимодействия

Топология сквозного проекта

```
Клиент Debian: curl http://127.0.0.1:8080/api/db-check
              |
              v
      [nginx-course7:80]
              |
              | HTTP proxy_pass
              v
        [api-net6:3000]
              |
              | PostgreSQL / TCP 5432
              v
         [pg-course:5432]
              |
              v
      [pgdata-course18]

api-net6, pg-course и nginx-course7 подключены к course-net
```

Nginx обращается к API по DNS-имени api-net6, а API — к PostgreSQL по имени pg-course. Сеть course-net уже создана в ЛР №6 и объявляется в Compose как external. Том pgdata-course18 создан в ЛР №5 и также объявляется как external.

6. Подготовка и проверка существующих объектов

Этап 1. Проверить Docker и Compose

Проверка Docker-среды

```
sudo docker version
sudo docker compose version
sudo docker ps -a
sudo docker volume ls
sudo docker network ls
sudo docker image ls
```

Критерий перехода: Docker Engine и Compose доступны. В списках должны присутствовать pgdata-course18, course-net, postgres:18 и express-pg:net6. Образ nginx:stable также должен быть локально либо доступен для загрузки.

Этап 2. Проверить данные PostgreSQL до перехода на Compose

Если pg-course остановлен, запустите его и убедитесь, что база shop содержит таблицу categories из ЛР №5.

Проверка существующего кластера

```
sudo docker start pg-course
sudo docker exec pg-course pg_isready -U student -d shop
sudo docker exec pg-course psql -U student -d shop  -c "SELECT id, name FROM categories ORDER BY id;"
```

Запишите фактическое количество строк и значения categories. Они понадобятся для сравнения после запуска через Compose. Количество строк может отличаться в зависимости от выполненного самостоятельного задания ЛР №5.

Этап 3. Проверить сеть, том и образы

Проверка сквозных объектов

```
sudo docker volume inspect pgdata-course18
sudo docker network inspect course-net
sudo docker image inspect postgres:18
sudo docker image inspect express-pg:net6
sudo docker image inspect nginx:stable
```

Критерий перехода: том и сеть существуют, а образы доступны. Если какого-либо объекта нет, не создавайте замену с другим именем: восстановите объект по соответствующей предыдущей лабораторной работе.

7. Передача контейнеров под управление Compose

Этап 4. Удалить только старые контейнеры

ВАЖНО	Контейнеры можно пересоздать, потому что данные PostgreSQL находятся в pgdata-course18. Volume и сеть удалять нельзя.

Освобождение имён контейнеров

```
sudo docker stop api-net6
sudo docker stop pg-course
sudo docker rm api-net6
sudo docker rm pg-course

sudo docker volume inspect pgdata-course18
sudo docker network inspect course-net
```

Если один из контейнеров уже остановлен, docker stop может сообщить, что он не запущен — это не проблема. После удаления контейнеров обязательно убедитесь, что pgdata-course18 и course-net сохранились.

Этап 5. Создать каталог Compose-проекта

Подготовка каталога

```
mkdir -p ~/compose-course7/nginx
cd ~/compose-course7
pwd
```

Путь	Назначение
compose.yaml	Описание сервисов и подключение внешних ресурсов
nginx/default.conf	Конфигурация reverse proxy Nginx

API-код и Dockerfile повторно не создаём: используется уже собранный в ЛР №6 образ express-pg:net6.

8. Конфигурация Nginx

Этап 6. Создать reverse proxy

Файл nginx/default.conf

```
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://api-net6:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Nginx пересылает запросы на api-net6:3000 внутри course-net. Использовать localhost:3000 нельзя: внутри контейнера Nginx localhost указывает на сам Nginx.

9. Описание compose.yaml

Этап 7. Создать Compose-конфигурацию

Содержимое compose.yaml

```
services:
  db:
    image: postgres:18
    container_name: pg-course
    environment:
      POSTGRES_DB: shop
      POSTGRES_USER: student
      POSTGRES_PASSWORD: Lab5-ChangeMe-2026
    volumes:
      - pgdata-course18:/var/lib/postgresql
    networks:
      course-net:
        aliases:
          - pg-course
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U student -d shop"]
      interval: 5s
      timeout: 3s
      retries: 10

  api:
    image: express-pg:net6
    container_name: api-net6
    environment:
      PGHOST: pg-course
      PGUSER: student
      PGPASSWORD: Lab5-ChangeMe-2026
      PGDATABASE: shop
    depends_on:
      db:
        condition: service_healthy
    networks:
      course-net:
        aliases:
          - api-net6

  nginx:
    image: nginx:stable
```

```
    container_name: nginx-course7
    ports:
      - "127.0.0.1:8080:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - api
    networks:
      - course-net

networks:
  course-net:
    external: true

volumes:
  pgdata-course18:
    external: true
```

Сервисы db и api — это имена внутри Compose, но фактические контейнеры сохраняют имена pg-course и api-net6 через container_name. API использует PGHOST=pg-course, как в ЛР №6. Сеть course-net и том pgdata-course18 объявлены external: Compose их использует, но не создаёт заново.

Этап 8. Проверить YAML до запуска

Проверка Compose-конфигурации

```
cd ~/compose-course7
sudo docker compose config
sudo docker compose config --services
```

Ожидаются сервисы db, api и nginx. Команда config должна завершиться без ошибок. Если Compose сообщает, что external network или external volume не найдены, вернитесь к проверкам предыдущих этапов и восстановите именно course-net или pgdata-course18.

10. Запуск и сквозная проверка

Этап 9. Запустить стек

Запуск Compose

```
sudo docker compose up -d
sudo docker compose ps
sudo docker ps
```

Ожидаются контейнеры pg-course, api-net6 и nginx-course7. Для PostgreSQL после инициализации healthcheck должен стать healthy. В этой работе API не собирается заново: Compose запускает существующий образ express-pg:net6.

Этап 10. Проверить сеть и том

Проверка переиспользования объектов

```
sudo docker network inspect course-net
sudo docker volume inspect pgdata-course18
sudo docker inspect pg-course --format '{{json .Mounts}}'
```

В course-net должны находиться pg-course, api-net6 и nginx-course7. У pg-course должен быть подключён volume pgdata-course18 с Destination=/var/lib/postgresql.

Этап 11. Проверить PostgreSQL 18

Проверка PGDATA, версии и данных

```
sudo docker compose exec db sh -c 'echo "$PGDATA"'
sudo docker compose exec db psql -U student -d shop -c "SHOW server_version;"
sudo docker compose exec db psql -U student -d shop  -c "SELECT id, name FROM categories ORDER BY id;"
```

Ожидается PGDATA=/var/lib/postgresql/18/docker и версия PostgreSQL 18.x. Таблица categories должна содержать те же строки, которые были зафиксированы до перехода на Compose.

Этап 12. Проверить Nginx → API → PostgreSQL

Сквозная проверка

```
curl -i http://127.0.0.1:8080/api/db-check
sudo docker compose logs --tail=30 nginx
sudo docker compose logs --tail=30 api
sudo docker compose logs --tail=30 db
```

Ожидается HTTP 200 и JSON вида {"ok":true,"categories":N}, где N — фактическое количество строк таблицы categories. Сравните N с прямым SQL-запросом. Так подтверждается путь curl → Nginx → api-net6 → pg-course → PostgreSQL.

11. Проверка постоянного хранения

Этап 13. Пересоздать Compose-контейнеры без удаления внешнего тома

Пересоздание и повторная проверка

```
sudo docker compose down
sudo docker volume inspect pgdata-course18
sudo docker network inspect course-net
sudo docker compose up -d
sudo docker compose exec db psql -U student -d shop  -c "SELECT id, name FROM categories ORDER BY id;"
curl -s http://127.0.0.1:8080/api/db-check
```

После docker compose down контейнеры текущего Compose-проекта удаляются, но external-ресурсы pgdata-course18 и course-net остаются. После повторного запуска данные categories должны сохраниться без изменений.

ВАЖНО	Не выполняйте docker volume rm pgdata-course18 и docker network rm course-net. Эти объекты принадлежат сквозному учебному проекту и нужны следующим лабораторным работам.

12. Диагностические эксперименты

Этап 14. Ошибка неправильного PGHOST

Сначала убедитесь, что корректная конфигурация работает. Затем временно измените в compose.yaml значение PGHOST у api с pg-course на nonexistent-db.

Намеренная DNS-ошибка

```
sudo docker compose up -d --force-recreate api
curl -i http://127.0.0.1:8080/api/db-check
sudo docker compose logs --tail=30 api
```

Ожидается HTTP 503 или другой отказ приложения, а в журнале api — ошибка разрешения имени или соединения. После фиксации результата верните PGHOST: pg-course.

Восстановление правильного PGHOST

```
sudo docker compose up -d --force-recreate api
curl -i http://127.0.0.1:8080/api/db-check
```

Этап 15. Остановка PostgreSQL

Диагностика остановленной базы и восстановление

```
sudo docker compose stop db
curl -i http://127.0.0.1:8080/api/db-check
sudo docker compose logs --tail=20 api

sudo docker compose start db
sudo docker compose exec db pg_isready -U student -d shop
curl -i http://127.0.0.1:8080/api/db-check
```

При остановленном PostgreSQL API остаётся запущенным, но SQL-запрос выполнить не может. После старта pg-course дождитесь accepting connections и повторите HTTP-проверку.

13. Самостоятельное задание

1. Измените публикацию Nginx с 127.0.0.1:8080:80 на 127.0.0.1:8081:80.
2. Примените изменения командой docker compose up -d.
3. Проверьте GET /api/db-check через порт 8081.
4. Выполните SELECT COUNT(*) FROM categories напрямую через docker compose exec db.
5. Сравните число categories в JSON и результат SQL-запроса.
6. Верните порт 8080 и подтвердите восстановление.
7. Убедитесь, что pgdata-course18 и course-net сохранили прежние имена.

14. Итоговая проверка

1. docker compose config выполняется без ошибок.
2. docker compose ps показывает сервисы nginx, api и db.
3. Фактические имена контейнеров: nginx-course7, api-net6 и pg-course.
4. Все три контейнера подключены к существующей сети course-net.
5. pg-course использует существующий volume pgdata-course18.
6. PostgreSQL работает на образе postgres:18.
7. API работает на образе express-pg:net6 и использует PGHOST=pg-course.
8. curl http://127.0.0.1:8080/api/db-check возвращает успешный JSON.
9. Количество categories совпадает с прямым SQL-запросом.
10. После docker compose down/up данные таблицы categories сохраняются.
11. После диагностических экспериментов стек полностью восстановлен.

15. Содержание отчёта

1. Тема, цель и схема сквозного проекта.
2. Таблица канонических имён Docker-объектов.
3. Результат проверки pgdata-course18 и course-net до запуска Compose.
4. Содержимое compose.yaml с external network и external volume.
5. Вывод docker compose config и docker compose ps.
6. Вывод docker network inspect course-net с тремя контейнерами.
7. Вывод docker volume inspect pgdata-course18 и Mounts контейнера pg-course.
8. Результат SELECT из categories до и после docker compose down/up.
9. Результат curl /api/db-check.
10. Один диагностический эксперимент: симптом, лог, причина и восстановление.
11. Результат самостоятельного задания.

16. Контрольные вопросы

1. Почему в ЛР №7 не создаётся новый Docker Volume для PostgreSQL?
2. Что означает external: true для сети и тома в Compose?
3. Почему перед первым docker compose up нужно удалить старые контейнеры pg-course и api-net6, но нельзя удалять pgdata-course18?

4. Чем имя Compose-сервиса db отличается от фактического имени контейнера pg-course?
5. Почему API использует PGHOST=pg-course, а не localhost?
6. Почему Nginx обращается к api-net6:3000, а не localhost:3000?
7. Нужно ли публиковать порт PostgreSQL 5432 на хосте для связи API с БД?
8. Что проверяет docker compose config?
9. Почему docker compose down не уничтожает external volume pgdata-course18?
10. Как доказать, что после пересоздания контейнеров используется прежний кластер PostgreSQL?
11. Как отличить ошибку DNS-имени контейнера от остановленной базы данных?
12. Какие команды помогают проверить сеть, том и журналы Compose-сервисов?

17. Критерии оценивания

Критерий	Баллы
Проверка и сохранение объектов ЛР №5–6	15
Корректный compose.yaml с external network/volume	20
Запуск Nginx, API и PostgreSQL с согласованными именами	20
Сквозная HTTP/SQL-проверка	15
Проверка сохранности данных после down/up	10
Диагностические эксперименты и восстановление	10
Отчёт и ответы на вопросы	10
Итого	100

После защиты стек можно оставить работающим для следующего занятия. Если нужно освободить контейнеры, выполните sudo docker compose down. Не удаляйте pgdata-course18 и course-net: это ресурсы сквозного проекта.